Project Requirements Document

Pizza Sales AI Analytics Agent

1. Executive Summary

This PRD specifies an AI-powered Analytics Agent that processes, analyzes, and visualizes pizza sales data. The system computes core business KPIs, generates charts, connects to the database through a secure read-only role, enforces structured routing with LangGraph, supports human-in-the-loop approval, maintains an audit trail, and is evaluated against a golden dataset.

2. Objectives and Scope

Build an agent that parses natural-language questions, maps them to the relational schema, executes queries under strict read-only constraints, and returns tabular metrics, visualizations, and auditable records.

3. Data Source

Table pizza_sales, 12 columns, about 48,620 rows (2015). Source dates are DD-MM-YYYY.

Column	Type	Note
pizza_id, order_id	INT	order_id repeats for multi-pizza orders
pizza_name_id, pizza_name	VARCHAR	Identifier and display name
quantity	INT	Pizzas on the row
order_date	DATE	Source format DD-MM-YYYY (2015 data)
order_time	TIME	HH:MM:SS
unit_price, total_price	NUMERIC(8,2)	total_price = unit_price x quantity
pizza_size, pizza_category	VARCHAR	Sizes S/M/L/XL/XXL; categories e.g. Classic, Veggie, Supreme
pizza_ingredients	TEXT	Comma-separated list

4. Key Performance Indicators

The agent must compute these KPIs dynamically. Because order_id repeats across rows and quantity can exceed 1, counts must use the definitions below.

KPI	Definition	SQL (table pizza_sales)
Total Revenue	Sum of total_price across all rows	SUM(total_price)
Total Orders	Count of distinct orders (order_id repeats across rows)	COUNT(DISTINCT order_id)
Total Pizzas Sold	Sum of quantity, not a row count	SUM(quantity)
Average Order Value	Total Revenue / Total Orders	SUM(total_price) / COUNT(DISTINCT order_id)
Avg Pizzas Per Order	Total Pizzas Sold / Total Orders	SUM(quantity) / COUNT(DISTINCT order_id)

5. Chart and Visualization Requirements

Chart	Type	Description and Purpose
Daily Trend, Total Orders	Bar	Orders per calendar date over a user-specified period (default: by date; day-of-week view optional) to show volume fluctuations.
Hourly Trend, Total Orders	Line	Orders by hour of day to pinpoint peak operating hours.
% of Sales by Pizza Category	Pie	Share of revenue (SUM(total_price)) per category.
% of Sales by Pizza Size	Pie	Share of revenue per size (S, M, L, XL, XXL).
Total Pizzas Sold by Category	Funnel (or bar)	SUM(quantity) per category, sorted descending. Categories are not sequential stages, so a bar chart is an acceptable alternative.
Top 5 Bestsellers	3 bar charts	One bar chart each for the top 5 pizzas by Revenue, by Total Quantity, and by Total Orders.

6. Technical Architecture and LangGraph Workflow

6.1 Linear Pipeline

Core flow: preflight → generate_sql → execute_sql → generate_chart → respond.

- Wiring: compiled via StateGraph. Use add_edge for the linear path and add_conditional_edges for every branch (out-of-scope bypass, confidence router, retry loop).
- Out-of-scope questions route from preflight directly to respond with a refusal.
- generate_chart is a conditional step after execute_sql: if the question maps to a Section 5 chart it builds the visualization, otherwise it routes straight to respond.
- execute_sql catches database errors into state (sql_error) rather than letting the graph crash.

6.2 Nodes

Node	Responsibility
preflight	Decides whether the question is answerable from the schema; out-of-scope questions route straight to respond with a polite refusal.
generate_sql	Produces SQL and a self-assessed confidence score via structured output; folds in human feedback or sql_error on retries.
human_review	Calls interrupt() with the SQL and reasoning; on resume, approves (to execute_sql) or rejects with a reason (back to generate_sql).
execute_sql	Runs the query on a read-only role connection; database errors are stored in state as sql_error instead of crashing the graph.
generate_chart	Builds the matching Section 5 chart from the query results when the question maps to one; otherwise passes through to respond.
respond	Returns tabular results, charts, refusals, or an honest failure message once retries are exhausted.

6.3 Routing, Human Approval, and Retry Cycle

- **Confidence self-assessment:** generate_sql returns a confidence score for its query using structured output.
- **Confidence router:** a conditional edge sends the graph to execute_sql if confidence ≥ CONFIDENCE_THRESHOLD, otherwise to the human_review node. interrupt() is called inside human_review, not in the edge.
- **Checkpointer:** the graph is compiled with a checkpointer (e.g. MemorySaver) and invoked with a thread_id in its config; interrupt() and resume require both.
- **Resume:** Command(resume=...) supports approval, or rejection with a reason that is folded into state and loops back to generate_sql.
- **Database error retry:** execution errors route back to generate_sql with sql_error in state for context-aware correction.
- **Retry bounds:** retry_count is bounded by MAX_RETRIES. Both database-error retries and human rejections count toward it (or document otherwise). Once exceeded, route to respond with an honest failure message.
- **Configuration:** CONFIDENCE_THRESHOLD and MAX_RETRIES are read from .env at import time, with no hardcoded constants.

6.4 Security and Audit

- Read-only database role, plus SELECT-only validation of generated SQL (reject INSERT, UPDATE, DELETE, DDL, multiple statements).
- Statement timeout and a row limit on every query.
- Audit trail: log the question, generated SQL, confidence score, reviewer decision and reason, retry count, and final result or error for every run.

7. Evaluation Framework

- Golden dataset: at least 8 natural-language questions, each paired with expected SQL or expected result rows. Include at least 1 to 2 out-of-scope questions to test the preflight refusal.
- Automated script: runs the graph over the set with scripted interrupt resumes (auto-approve, or auto-reject with canned reasons).
- Metrics reported: execution accuracy against expected results, retry frequency, and human-escalation frequency.